



**UNIVERSIDAD AUTÓNOMA DEL
ESTADO DE MÉXICO
FACULTAD DE INGENIERÍA**



Programa educativo.
Ingeniería en computación.

Unidad de aprendizaje:
Proyecto integral de comunicación de datos.

Grupo: 01.

Profesor:
Juan Carlos Escobar González.

Primer registro de avances proyecto.
Plataforma inteligente de monitoreo de redes

Equipo de trabajo:
Daniel Dávila Lara.
Jesús Omar Carranza Colín.

Fecha de entrega

26/Marzo/2026

PALABRA: GENERACION

1 Requerimientos Funcionales y No Funcionales

1.1 Requerimientos Funcionales

Fecha completada: 24 de febrero de 2026

ID	Requerimiento	Descripción
01	Captura de tráfico	El sistema debe capturar paquetes de red en tiempo real usando Scapy.
02	Filtrado de paquetes	El sistema debe aplicar filtros para seleccionar solo paquetes dependiendo de los protocolos TCP, UDP, ICMP, HTTP relevantes.
03	Extracción de atributos	Por cada paquete capturado se deben extraer: IP origen, IP destino, puerto origen, puerto destino, protocolo.
04	Detección de anomalías	El sistema debe clasificar cada paquete como 'normal' o 'anómalo'.
05	Generación de alertas	Al detectar una anomalía, el sistema debe generar una alerta dependiendo la anomalía
06	Almacenamiento de eventos	Todos los eventos deben guardarse en un archivo de logs o si es posible una base de datos.
07	Dashboard de visualización	El sistema debe proveer una interfaz web que muestre: anomalías por protocolo, así como la lista de alertas.
08	Integración de módulos	El módulo de captura debe comunicarse con el módulo de detección mediante una cola de mensajes.
09	Entorno IoT	El sistema debe operar con al menos 2 dispositivos ESP32 conectados a una red LAN de laboratorio generando tráfico real.

1.2 Requerimientos No Funcionales

ID	Requerimiento	Descripción
01	Rendimiento	El sistema debe procesar y clasificar cada paquete en ≤ 500 ms.
02	Disponibilidad	El sistema debe capturar tráfico de forma continua durante la sesión de prueba sin interrupciones no controladas.
03	Mantenibilidad	El código fuente debe seguir un estándar de documentación de funciones importantes para futuras actualizaciones.
04	Software libre	El sistema debe desarrollarse con herramientas open source sin licencias de pago.
05	Compatibilidad	El sistema debe ejecutarse en Linux o en Windows con Python 3.10+ sin configuraciones adicionales.
06	Seguridad básica	El dashboard debe funcionar con estándares de seguridad como encriptación y autenticación básica para los usuarios.

2 Criterios de Aceptación

Fecha completada: 26 de febrero de 2026

Los criterios de aceptación definen cómo vamos a comprobar que cada requerimiento funciona correctamente. Para cada uno se indica qué se prueba, cómo se prueba y qué resultado se espera.

ID Req.	¿Qué se verifica?	¿Cómo se comprueba?	¿Qué resultado se espera?
01 Captura	El sistema captura paquetes de la red en tiempo real.	Se genera tráfico con los ESP32 y se observan los logs de captura.	Los paquetes aparecen registrados en el log en menos de 1 segundo desde que se generan.
02 Filtrado	Solo se procesan paquetes de los protocolos seleccionados (TCP, UDP, ICMP, HTTP).	Se envía tráfico de distintos protocolos y se revisa que solo los esperados aparezcan en los logs.	No aparecen paquetes de protocolos no incluidos en el filtro.
03 Atributos	El sistema extrae correctamente IP origen, IP destino, puerto origen, puerto destino y protocolo de cada paquete.	Se revisan los registros generados para 100 paquetes capturados.	Los 5 campos están presentes y con valores correctos en al menos el 95% de los paquetes.
04 Detección	El modelo clasifica correctamente tráfico normal y tráfico anómalo.	Se ejecuta el modelo con muestras conocidas de tráfico normal y anómalo.	La mayoría de las muestras se clasifican correctamente (precisión $\geq 80\%$).
05 Alertas	Se genera una alerta cada vez que se detecta tráfico anómalo.	Se simula tráfico anómalo con hping3 y se revisa el registro de alertas.	Cada evento anómalo genera al menos una alerta visible en el sistema.
06 almacenamiento	Los eventos y alertas quedan guardados para poder consultarlos después.	Se revisa el archivo de logs o la base de datos después de una sesión de prueba.	Todos los eventos registrados durante la sesión están disponibles al consultar el archivo o BD.
07 Dashboard	La interfaz web muestra las anomalías por protocolo y la lista de alertas.	Se accede al dashboard desde el navegador después de generar tráfico de prueba.	El dashboard carga correctamente, muestra datos actualizados y la lista de alertas es visible.
08 Integración	Los módulos de captura y detección se comunican sin perder datos.	Se captura tráfico durante 5 minutos y se compara el número de paquetes capturados con los analizados.	La diferencia entre paquetes capturados y analizados es menor al 5%.
09 Entorno IoT	Los ESP32 generan tráfico visible en la red del laboratorio.	Se verifica la conectividad de ambos ESP32 con un ping y se observa su tráfico en el dashboard.	Ambos dispositivos responden al ping y su tráfico aparece en los registros del sistema.

3 Arquitectura del Sistema

Fecha completada: 03 de marzo al 06 de marzo de 2026

El sistema NetGuard sigue una arquitectura en capas con flujo de datos unidireccional: los datos van de la red física hacia el motor de análisis y finalmente hacia la capa de presentación.

Cada capa es independiente y se comunica con la siguiente a través de módulos definidos.

Descripción general de la arquitectura

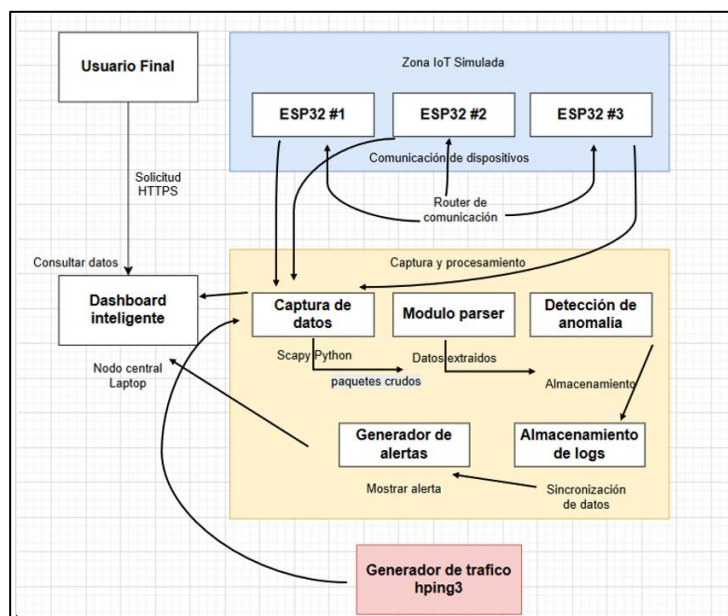
El sistema se compone de 4 capas principales que corresponden a las zonas del diagrama lógico:

- Capa de Red / Ecosistema IoT
- Capa de Captura y Preprocesamiento
- Capa de Análisis y clasificación de anomalías
- Capa de Presentación (Dashboard)

Capa	Componentes	Tecnología	Responsabilidad principal
1. Red / IoT	ESP32 #1, ESP32 #2, Red LAN, Generador tráfico (hping3)	Hardware + Arduino IDE	Generar tráfico de red real y simulado en una red LAN
2. Captura	Detector Scapy, Filtro de paquetes, Limpieza de paquetes	Python 3.10 + Scapy	Capturar, filtrar y procesar paquetes
3. Análisis	Umbral de decisión para generación de alertas	Python + librerías extras	Clasificar tráfico como normal/anómalo; generar alertas y guardarlas en logs.
4. Aplicación	API REST Flask, Dashboard HTML/JS	Flask + HTML, JS	Mostrar el análisis de paquetes web con gráficas y lista de alertas

Principios de diseño aplicados

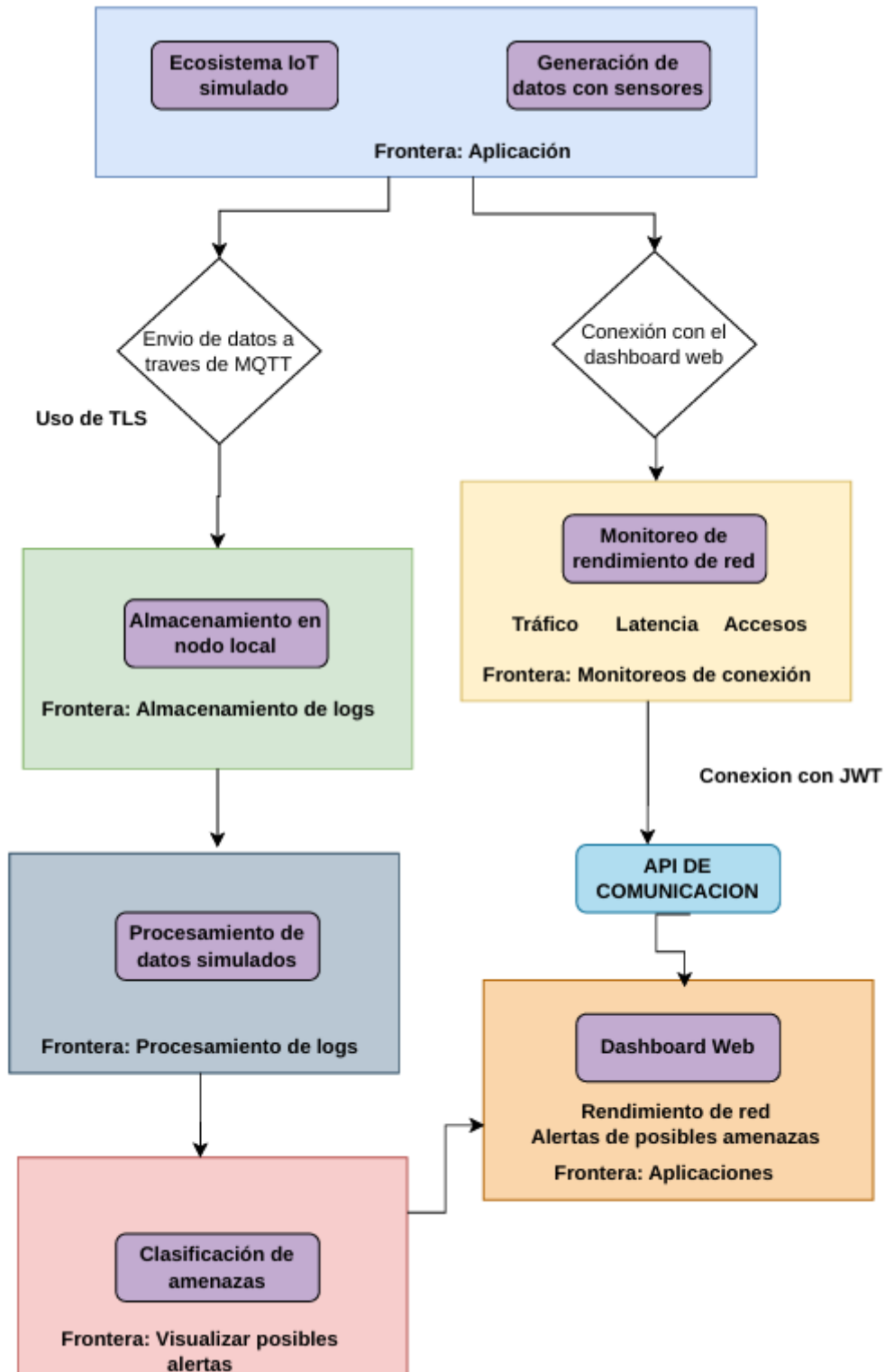
- Separación de responsabilidades: cada módulo hace una sola cosa.
- Persistencia local: Archivo de logs de los paquetes recibidos.
- Software libre: Herramientas open source para el análisis de paquetes.
- Seguridad por capas: Filtros de clasificación de amenazas, validación de paquetes provenientes de Scapy, seguridad de autenticación en dashboard.



**Diagrama de
diseño lógico**
19 de marzo 2026

4 Diagrama de Flujo de Datos

Fecha completada: 17 de marzo de 2026



5 Identificación y Direccionamiento IP

Fecha completada: 19 de marzo de 2026

El sistema opera en una red de laboratorio privada de clase C con un límite de

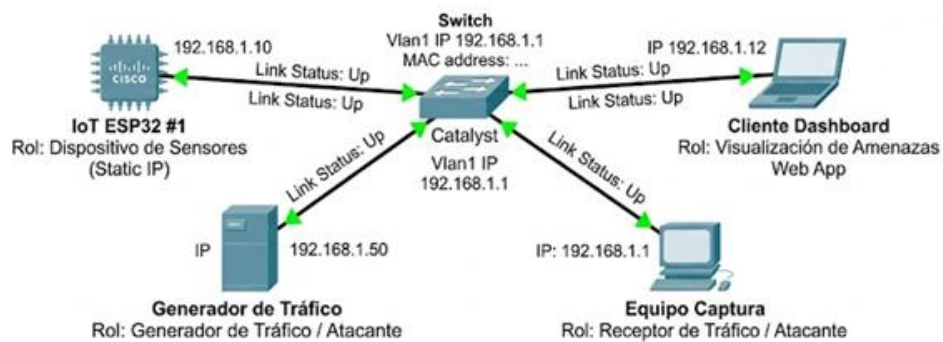
Tabla de asignación de IPs fijas por dispositivo

Dispositivo	Rol	IP Asignada	Máscara	Notas
ESP32 #1	Dispositivo IoT	192.168.1.10	/24	IP estática configurada en Arduino
ESP32 #2	Dispositivo IoT	192.168.1.11	/24	IP estática configurada en Arduino
Equipo Servidor	Generador de tráfico	192.168.1.50	/24	Máquina de generación de tráfico.
Equipo Captura	Receptor de tráfico	192.168.1.1	/24	Máquina de recepción de paquetes.

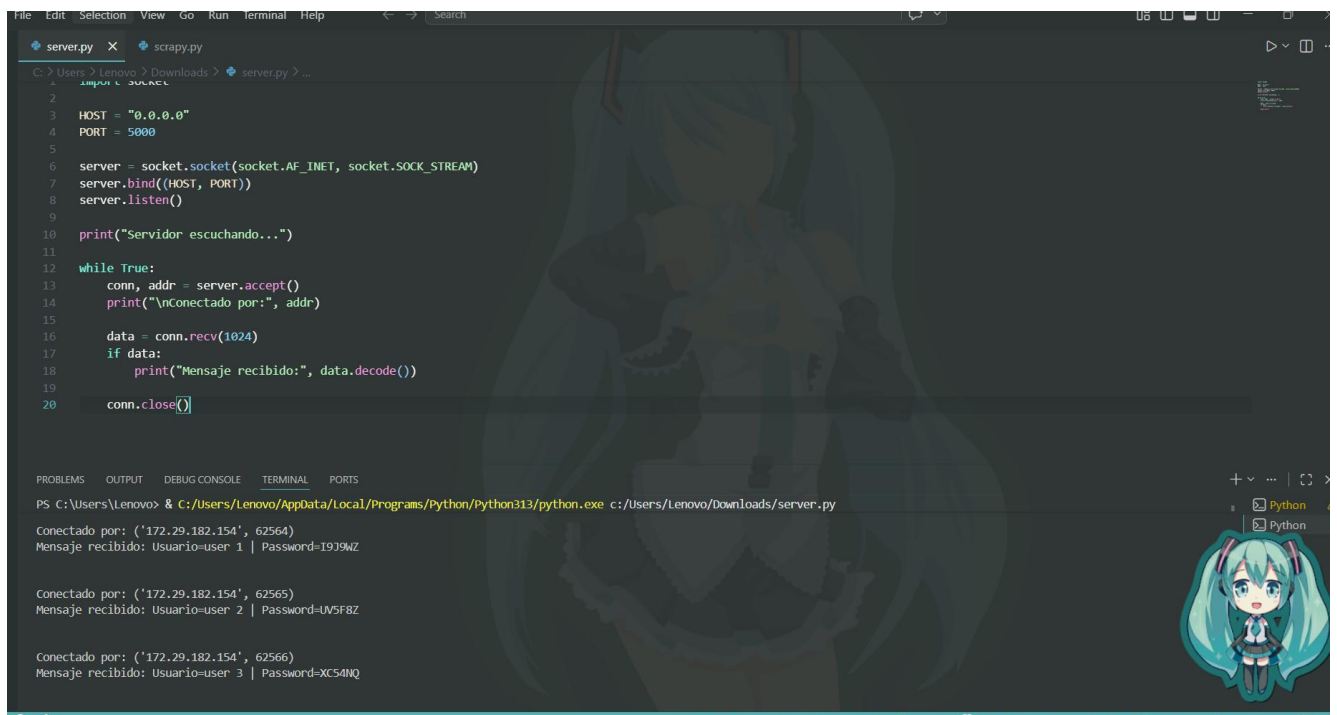
6 Diagrama de Topología de Red

Fecha completada: 23 de marzo de 2026

El sistema opera en una red de laboratorio privada de clase C con un límite de 254 hosts (Capa 3: 192.168.1.0/24).



7 Entorno de red simulado



The screenshot shows a Windows terminal window with a dark background. The terminal is running a Python script named `server.py`. The script is a simple TCP server that listens on port 5000 and prints received data. The output shows three connections from 172.29.182.154 with different passwords.

```
server.py
1 import socket
2
3 HOST = "0.0.0.0"
4 PORT = 5000
5
6 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
7 server.bind((HOST, PORT))
8 server.listen()
9
10 print("Servidor escuchando...")
11
12 while True:
13     conn, addr = server.accept()
14     print("\nConectado por:", addr)
15
16     data = conn.recv(1024)
17     if data:
18         print("Mensaje recibido:", data.decode())
19
20     conn.close()
```

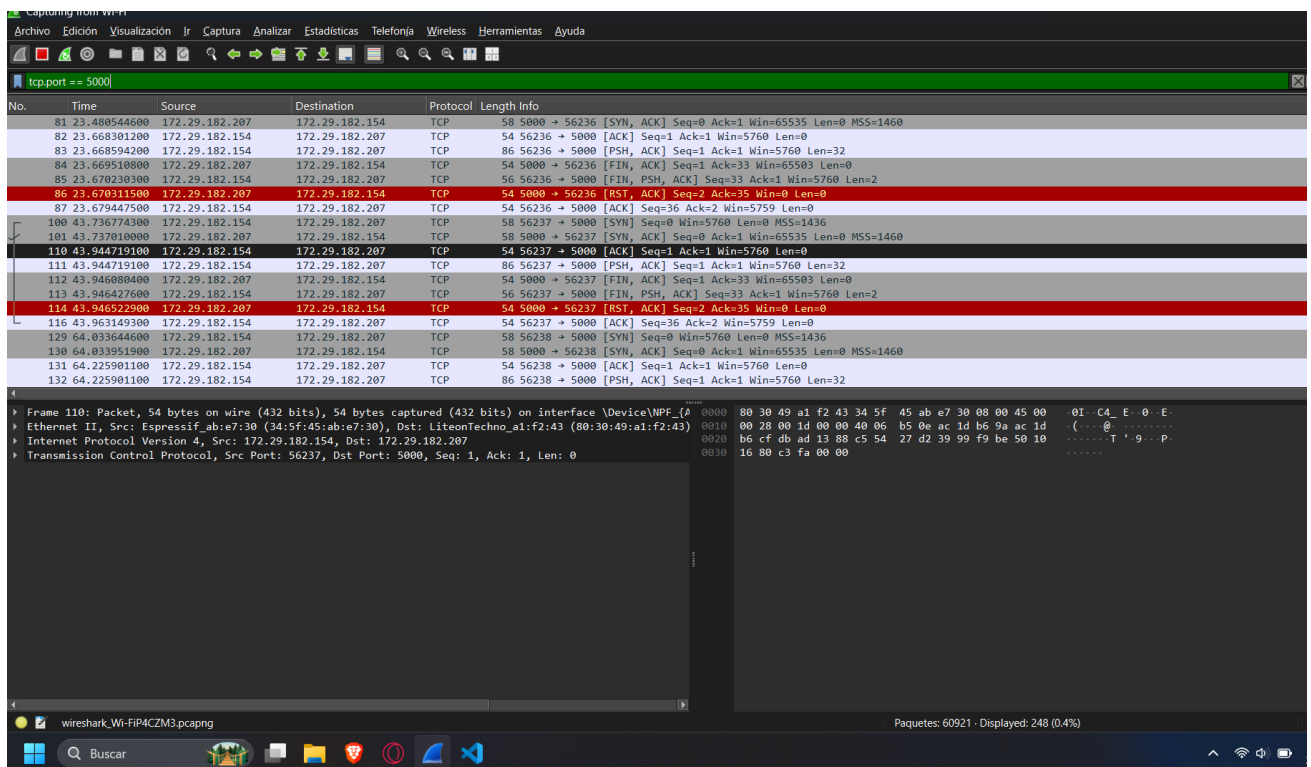
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Lenovo> & C:\Users\Lenovo\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Lenovo/Downloads/server.py

Conectado por: ('172.29.182.154', 62564)
Mensaje recibido: Usuario=user 1 | Password=I939WZ

Conectado por: ('172.29.182.154', 62565)
Mensaje recibido: Usuario=user 2 | Password=UV5F8Z

Conectado por: ('172.29.182.154', 62566)
Mensaje recibido: Usuario=user 3 | Password=XC54NQ



The screenshot shows a Wireshark network traffic capture. The filter is `tcp.port == 5000`. The table below shows the captured packets, highlighting the connection establishment and data transfer.

No.	Time	Source	Destination	Protocol	Length	Info
81	23.480544600	172.29.182.207	172.29.182.154	TCP	58	5000 → 56236 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460
82	23.668301200	172.29.182.154	172.29.182.207	TCP	54	56236 → 5000 [ACK] Seq=1 Ack=1 Win=5760 Len=0
83	23.668594200	172.29.182.154	172.29.182.207	TCP	86	56236 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=5760 Len=32
84	23.669510800	172.29.182.207	172.29.182.154	TCP	54	5000 → 56236 [FIN, ACK] Seq=1 Ack=33 Win=65503 Len=0
85	23.670230300	172.29.182.154	172.29.182.207	TCP	56	56236 → 5000 [FIN, PSH, ACK] Seq=33 Ack=1 Win=5760 Len=2
86	23.670311500	172.29.182.207	172.29.182.154	TCP	54	5000 → 56236 [RST, ACK] Seq=2 Ack=35 Win=0 Len=0
87	23.679447500	172.29.182.154	172.29.182.207	TCP	54	56236 → 5000 [ACK] Seq=36 Ack=2 Win=5759 Len=0
100	43.736774300	172.29.182.154	172.29.182.207	TCP	58	56237 → 5000 [SYN] Seq=0 Win=5760 Len=0 MSS=1436
101	43.737010000	172.29.182.207	172.29.182.154	TCP	58	5000 → 56237 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460
110	43.944719100	172.29.182.154	172.29.182.207	TCP	54	56237 → 5000 [ACK] Seq=1 Ack=1 Win=5760 Len=0
111	43.944719100	172.29.182.154	172.29.182.207	TCP	86	56237 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=5760 Len=32
112	43.946080400	172.29.182.207	172.29.182.154	TCP	54	5000 → 56237 [FIN, ACK] Seq=1 Ack=33 Win=65503 Len=0
113	43.946427600	172.29.182.154	172.29.182.207	TCP	56	56237 → 5000 [FIN, PSH, ACK] Seq=33 Ack=1 Win=5760 Len=2
114	43.946522900	172.29.182.207	172.29.182.154	TCP	54	5000 → 56237 [RST, ACK] Seq=2 Ack=35 Win=0 Len=0
116	43.963149300	172.29.182.154	172.29.182.207	TCP	54	56237 → 5000 [ACK] Seq=36 Ack=2 Win=5759 Len=0
129	64.033644600	172.29.182.154	172.29.182.207	TCP	58	56238 → 5000 [SYN] Seq=0 Win=5760 Len=0 MSS=1436
130	64.033951900	172.29.182.207	172.29.182.154	TCP	58	5000 → 56238 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460
131	64.225901100	172.29.182.154	172.29.182.207	TCP	54	56238 → 5000 [ACK] Seq=1 Ack=1 Win=5760 Len=0
132	64.225901100	172.29.182.154	172.29.182.207	TCP	86	56238 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=5760 Len=32

Frame 110: Packet, 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface \Device\NPF_{...}

Ethernet II, Src: Expressif ab:e7:30 (34:5f:45:ab:e7:30), Dst: LiteonTechno_a1:f2:43 (80:30:49:a1:f2:43)

Internet Protocol Version 4, Src: 172.29.182.154, Dst: 172.29.182.207

Transmission Control Protocol, Src Port: 56237, Dst Port: 5000, Seq: 1, Ack: 1, Len: 0

